Name: [Type your Complete Name Here]

Course & Year: BS Information Technology, [Type your Year Here]

Step 3: Documentation

Request	Method	Status Code	Response Summary
Get all posts	GET	200 OK	Returns a JSON array containing 100 post objects. Each object includes a userId, id, title, and body.
Get post 1	GET	200 OK	Returns a single JSON object representing the specific post with id: 1, containing its respective details.
Create post	POST	201 Created	Returns the newly created post object in JSON format, echoing back the submitted data along with a newly assigned id (e.g., id: 101).
Update post	PUT	200 OK	Returns the updated JSON object for post 1, reflecting the modified title, body, or userId submitted in the request body.
Delete post	DELETE	200 OK	Returns an empty JSON object {}, indicating that the post was successfully deleted from the server.

Reflection

What did you notice about the responses from the API?

The API responses are strictly structured in JSON format, making them easy to parse programmatically. Additionally, the API provides appropriate HTTP status codes to indicate the result of the operation (e.g., returning 200 OK for successful retrievals and updates, and 201 Created when successfully making a POST request). Because JSONPlaceholder is a mock API, requests that modify data (POST, PUT, DELETE) return successful responses as if the operation worked, but the changes aren't actually saved to their database permanently.

Which HTTP method did you find easiest to understand? Why?

GET was the easiest to understand because it simply involves requesting data from a specific endpoint. It does not require formatting a JSON payload or configuring headers in the request body like POST or PUT do. You simply point Postman (or even a standard web browser) to the URL, and the server returns the requested data immediately.

Why are REST APIs important in modern web development?

REST APIs are crucial because they establish a standardized way for different software systems to communicate over the internet. By separating the frontend (client) from the backend (server), developers can build scalable architectures. This means a single REST API can serve data simultaneously to multiple platforms—such as a full-stack web application, a mobile UI, or a terminal-based administrative tool—without needing to rewrite the core backend logic for each one.